

NPM

INFO

Lesinhoud

Kennismaking met NPM (Node Package Manager) en het gebruik ervan om een Vue-project te genereren worden behandeld.

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.

▼ Inhoud

- [Wat is NPM?](#)
- [Installeren van NPM](#)
 - [Bevestigen dat NPM geïnstalleerd is](#)
- [Werken met NPM](#)
 - [Een eerste project aanmaken](#)
- [package.json](#)
 - [Scripts](#)
 - [Dependencies](#)
- [node_modules](#)
- [NPM Create - Vue](#)

- [Folderstructuur Vue-project](#)
- [Extra opties - Linter & Prettier](#)
 - [Linter](#)
 - [Prettier](#)

Wat is NPM?

NPM staat voor **N**ode **P**ackage **M**anager. Een platform om software te delen en te hergebruiken.

De **registry** van NPM vind je op <https://www.npmjs.com/>. Hierop kan je software zoeken die je in projecten kan gebruiken, bijvoorbeeld [Vue.js](#).

Installeren van NPM

NPM wordt samen met [Node.js](#) geïnstalleerd.

Bezoek de website, klik op **Download** en kies vervolgens voor:

- **Get Node.js:** Kies de versie waar **LTS** bijstaat.
 - **LTS** staat voor **L**ong **T**erm **S**upport. Dit is de stabiele versie die lang ondersteund zal worden.
- **for:** Kies hier het besturingssysteem (Windows, macOS, Linux, ...)
- **using:** Kies hier de manier waarop je Node.js installeert.
 - Windows: **chocolatey**
 - macOS: **nvm**
 - Dit staat voor **N**ode **V**ersion **M**anager. Hiermee kan je verschillende versies van Node.js naast elkaar installeren en gebruiken.
 - Linux: **nvm**

- with: `npm`

Voer de commando's uit die op de website staan om Node.js te installeren.

Bevestigen dat NPM geïnstalleerd is

Open een terminal en voer het volgende commando uit:

```
npm -v
```

sh

Wanneer een versienummer verschijnt, heb je NPM correct geïnstalleerd.

Werken met NPM

NPM is een CLI-tool (**C**ommand **L**ine **I**nterface) die je in een terminal kan gebruiken. Met NPM maak je in een `map` (directory) een `project` aan en beheer je deze.

In dit opleidingsonderdeel werk je vanuit de gebruikersmap (`~`), daarin maak je een map `projects` aan. Binnen deze map plaats je een map `web-advanced` voor alle projecten van dit opleidingsonderdeel.

Maak deze mappen aan via Finder/Verkenner of via onderstaand commando.

```
mkdir -p ~/projects/web-advanced
```

sh

Navigeer via de terminal naar de map `web-advanced`.

```
cd ~/projects/web-advanced
```

sh

Een eerste project aanmaken

Maak in `web-advanced` een map `first-npm-project` aan en navigeer ernaartoe.

```
mkdir first-npm-project
```

sh

```
cd first-npm-project
```

sh

Een NPM-project maak je aan met het commando `npm init`. Dit commando maakt een `package.json`-bestand aan in de huidige map. Dit bestand bevat alle informatie over het project.

```
npm init -y
```

sh

Het `-y`-argument zorgt ervoor dat je alle standaardwaarden gebruikt. Zonder dit argument stelt NPM je een aantal vragen om het `package.json`-bestand aan te maken.

Bekijk wat er in de map staat.

```
ls
```

sh

Er zou nu een bestand `package.json` aanwezig moeten zijn.

Open `first-npm-project` in de editor en bekijk de inhoud van `package.json`.

package.json

```
{  
  "name": "first-npm-project",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

json

In dit JSON-bestand voeg je alle informatie over het project in. De licentie waaronder je de code van het project plaatst, geef je aan met `license`. De `ISC` -licentie is een open-source-licentie, een andere veelgebruikte open-source-licentie is de `MIT` -licentie. Wanneer je code publiek deelt, is het belangrijk om een licentie toe te voegen die aangeeft wat met de code mag worden gedaan. Ook is het belangrijk om te weten onder welke licentie je de code van anderen mag gebruiken.

Scripts

In het `scripts` -object definieer je commando's die je later met `npm run <commando>` kan uitvoeren, waarbij `<commando>` de naam is van het script dat je wilt uitvoeren.

In het standaardbestand staat er een script `test` dat een `echo` -commando uitvoert. Dit commando toont een bericht in de terminal.

```
npm run test
```

sh

Output:

```
> first-npm-project@1.0.0 test
> echo "Error: no test specified" && exit 1
Error: no test specified
```

De eerste lijn geeft aan welk script je uitvoert. De tweede lijn geeft aan wat het script doet. De derde lijn is de output van het commando dat je hebt uitgevoerd.

▼ Oefening

Voeg een nieuw script `hello` toe dat het bericht `Hello, World!` toont in de terminal. In plaats van `exit 1` kan `exit 0` worden gebruikt om aan te geven dat het script succesvol is uitgevoerd. (`0` staat voor succesvol, `1` staat voor een fout)

Test het script uit met `npm run hello`.

▼ Oplossing

```
{
  "name": "first-npm-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "hello": "echo \"Hello, World!\" && exit 0"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "chalk": "^5.6.2"
  }
}
```

json

```
}  
}
```

De scripts kunnen per project verschillen, bijvoorbeeld bij een Vue-project kunnen er scripts zijn om de **development server** te starten, de code te **builden** voor productie, ...

Dependencies

In projecten kan je code gebruiken die anderen hebben geschreven. Deze code noemt men **dependencies** (Nederlands: afhankelijkheden). Je installeert deze code via NPM.

Als voorbeeld maak je een JavaScript-bestand aan in het project, **index.js**.

```
console.log("npm werkt!");  
console.log("We hebben chalk geïnstalleerd 🌈");
```

js

Je voert dit bestand uit met Node.js.

```
node index.js
```

sh

Om de output kleurrijker te maken zal software van derden worden gebruikt. Het pakket (de dependency) **chalk** wordt hiervoor gebruikt.

```
console.log(chalk.green("npm werkt!"));  
console.log(chalk.blue.bold("We hebben chalk  
geïnstalleerd 🌈"));
```

js

Voer opnieuw het bestand uit.

```
node index.js
```

sh

Er verschijnt een foutmelding omdat `chalk` niet beschikbaar is. Je installeert `chalk` via NPM om de foutmelding op te lossen.

```
npm install chalk
```

sh

Dit commando doet verschillende dingen:

- Het downloadt het pakket `chalk` van de NPM-registry en plaatst het in een map genaamd `node_modules` in de huidige map.
- Het voegt `chalk` toe aan het `dependencies` -object in het `package.json` -bestand.

De map `node_modules` zal alle code bevatten die nodig is om `chalk` te gebruiken, inclusief alle dependencies die `chalk` zelf nodig heeft.

`chalk` kan nu in `index.js` geïmporteerd worden.

```
import chalk from "chalk";

console.log(chalk.green("npm werkt!"));
console.log(chalk.blue.bold("We hebben chalk
geïnstalleerd 🎉"));
```

js

Voer opnieuw het bestand uit.

```
node index.js
```

sh

Nu wordt de output in kleur getoond. Mogelijk verschijnt er een `Warning`. Dit komt doordat `import` (ES module) wordt gebruikt in plaats van `require` (CommonJS). Dit is een verschil in modulesystemen

binnen JavaScript. Voorlopig is dit niet van belang, als oplossing kan in `package.json` het volgende worden toegevoegd:

```
"type": "module",
```

json

Dus `package.json` wordt dan:

```
{
  "name": "first-npm-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "hello": "echo \"Hello, World!\" && exit 0"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "chalk": "^5.6.2"
  }
}
```

json

Het kan zijn dat de versie van `chalk` anders is in jouw `package.json`, dat is oké.

Voer opnieuw het bestand uit.

```
node index.js
```

sh

De output zou nu in kleur moeten worden getoond zonder `Warning`.

▼ Oefening

Voeg een script toe genaamd `start` dat het `index.js` -bestand uitvoert met Node.js. Test het script uit met `npm run start`.

Verwachte output:

```
> first-npm-project@1.0.0 start
> node index.js
npm werkt!
We hebben chalk geïnstalleerd 🍷
```

sh

Waarbij de console.log-berichten in kleur worden weergegeven.

▼ Oplossing

```
{
  // hier nog code
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "hello": "echo \"Hello, World!\" && exit 0",
    "start": "node index.js"
  }
  // hier nog code
}
```

json

▼ Oefening

Bekijk de documentatie van [chalk](#) en pas `index.js` aan zodat er een extra console.log-bericht wordt getoond met de tekst `Ik heb documentatie gelezen!`. De tekst heeft een witte achtergrond en staat schuin gedrukt (italic).

Test het bestand uit met `npm run start`. (indien dit niet werkt, voer dan eerst de vorige oefening uit)

▼ Oplossing

```
import chalk from "chalk";

console.log(chalk.green("npm werkt!"));
console.log(chalk.blue.bold("We hebben chalk
geïnstalleerd 🎉"));
console.log(chalk.bgWhite.italic("Ik heb documentatie
gelezen!"));
```

js

node_modules

De map `node_modules` bevat **alle code** die je nodig hebt om de dependencies te gebruiken in het project. Deze map kan dus heel groot worden. Daarom is het belangrijk om deze map **niet op te nemen in versiebeheer**.

Voeg een `.gitignore` -bestand toe aan de map `first-npm-project` met volgende inhoud:

```
node_modules
```

sh

De reden dat je `node_modules` niet op hoeft te nemen in versiebeheer is dat je de dependencies altijd opnieuw kan installeren via `npm install`. Dit commando leest het `package.json` -bestand en installeert de dependencies die daarin staan.

NPM Create - Vue

Met NPM genereer je projecten die gebaseerd zijn op een `template`. Dit doe je met het commando `npm create <package>`, waarbij `<package>` de naam is van de software waarvoor je een project wilt genereren.

Navigeer naar de map `web-advanced`.

```
cd ~/projects/web-advanced
```

sh

Met `npm create` kan een Vue-project gegenereerd worden.

Informatie kan afwijken

De informatie in de volgende stappen kan afwijken van wat er in de terminal verschijnt. Dit komt doordat de software regelmatig geüpdatet wordt, waardoor er mogelijk nieuwe versies of extra opties zijn. De algemene stappen en concepten blijven echter hetzelfde.

```
npm create vue@latest
```

sh

Er verschijnen enkele vragen.

```
Need to install the following packages:  
create-vue@3.18.1  
Ok to proceed? (y)
```

sh

Druk op `Enter`. Merk op dat de versie van `create-vue` kan verschillen.

```
┌ Vue.js - The Progressive JavaScript Framework  
│  
◇ Project name (target directory):  
│ first-vue-project  
│
```

sh

Project name (target directory) : dit is de naam van de map die je gaat aanmaken voor het project. Voer hier **first-vue-project** in en druk op **Enter**.

```
◆ Select features to include in your project: (↑/↓ to
navigate, space to select, a to toggle all, enter to
confirm)
```

- | ☐ TypeScript
- | ☐ JSX Support
- | ☐ Router (SPA development)
- | ☐ Pinia (state management)
- | ☐ Vitest (unit testing)
- | ☐ End-to-End Testing
- | ☐ Linter (error prevention)
- | ☐ Prettier (code formatting)

sh

Laat alle opties voorlopig uitgeschakeld en druk op **Enter**. Sommige opties zul je later inschakelen om te bekijken wat ze doen.

```
◆ Select experimental features to include in your
project: (↑/↓ to navigate, space to select, a to
toggle all, enter to confirm)
| ☐ Replace Prettier with Oxfmt
| ☐ Vite 8 (beta)
| ☐ Vue 3.6 (beta)
```

sh

Laat alle opties uitgeschakeld en druk op **Enter**.

```
◆ Skip all example code and start with a blank Vue
project?
| ☐ Yes / ☒ No
```

sh

Laat **No** geselecteerd en druk op **Enter**.

```
Scaffolding project in /hier-jouw-user/Projects/web-advanced/first-vue-project...
```

sh

```
|  
└─ Done. Now run:
```

```
cd first-vue-project  
npm install  
npm run dev
```

```
| Optional: Initialize Git in your project directory  
with:
```

```
git init && git add -A && git commit -m "initial  
commit"
```

Het optionele sla je voorlopig over - dit komt later aan bod wanneer je projecten deelt via GitHub.

Zoals de CLI aangeeft heb je het project `first-vue-project` aangemaakt. Vervolgens navigeer je naar die map en installeer je de dependencies.

```
cd first-vue-project
```

sh

Installeer de dependencies.

```
npm install
```

sh

Start de development server.

```
npm run dev
```

sh

Alles wat je met `npm run` uitvoert, zijn scripts die in `package.json` zijn gedefinieerd. Je kan het `package.json` -bestand bekijken om te zien welke scripts aanwezig zijn.

```
{  
  // hier nog code  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "preview": "vite preview"  
  }  
  // hier nog code  
}
```

json

Bij het uitvoeren van `npm run dev` voer je dus het commando `vite` uit. Vite is een tool die je binnen moderne webontwikkeling gebruikt om een snelle development server te bieden en de code te bundelen voor productie.

Bij het uitvoeren van `npm run build` bundel je de code voor productie. Dit betekent dat je alle code samenbrengt, optimaliseert en klaarmaakt om op een webserver te plaatsen. Bij het uitvoeren van `npm run preview` start je een lokale server om de gebundelde code te bekijken. Op deze manier kan je testen hoe de website er in productie zal uitzien.

Bij het uitvoeren van `npm run dev` start je een lokale server en verschijnt er een URL (meestal `http://localhost:5173/`), die je in een browser kan openen om de Vue-app te bekijken. De server herlaadt automatisch wanneer je wijzigingen in de code aanbrengt. De URL die in de terminal verschijnt na het uitvoeren van `npm run dev` kan je in de browser openen. Afhankelijk van de instellingen van het besturingssysteem (dark mode of light mode) wordt de pagina in het zwart of wit weergegeven.

Folderstructuur Vue-project

```
first-vue-project/  
- .vscode/  
- node_modules/  
- public/  
- src/  
.gitignore  
index.html  
package.json  
package-lock.json  
README.md  
vite.config.js  
jsconfig.json
```

plaintext

- **.vscode/** : Configuratiebestanden voor Visual Studio Code, indien deze editor gebruikt wordt.
- **node_modules/** : Bevat alle geïnstalleerde dependencies voor het project.
- **public/** : Bevat statische bestanden die direct gekopieerd worden naar de root van de build-output.
- **src/** : Bevat alle broncode van de applicatie, inclusief componenten, stijlen en andere assets.
- **.gitignore** : Bestanden en mappen die genegeerd worden door Git.
- **index.html** : Het HTML-bestand dat geladen wordt waarin de Vue-app wordt gekoppeld (gemount).
- **package.json** : Bevat metadata over het project, scripts en een lijst van dependencies.
- **package-lock.json** : Bevat een gedetailleerde lijst van alle geïnstalleerde dependencies en hun versies, om consistentie te garanderen bij het installeren van dependencies.
- **README.md** : Een Markdown-bestand dat informatie over het project bevat.

- `vite.config.js` : Configuratiebestand voor Vite, de build tool die gebruikt wordt in dit project.
- `jsconfig.json` : Configuratiebestand voor JavaScript-projecten, gebruikt door editors zoals Visual Studio Code om de projectstructuur en module-resolutie te begrijpen.

Extra opties - Linter & Prettier

Genereer een nieuw Vue-project en kies ditmaal enkele extra opties.

```
npm create vue@latest
```

sh

Kies als projectnaam `vue-extra`.

Schakel de volgende opties in:

```
◆ Select features to include in your project: (↑/↓ to
navigate, space to select, a to toggle all, enter to
confirm)
```

```
| ☐ TypeScript
| ☐ JSX Support
| ☐ Router (SPA development)
| ☐ Pinia (state management)
| ☐ Vitest (unit testing)
| ☐ End-to-End Testing
| ☒ Linter (error prevention)
| ☒ Prettier (code formatting)
```

sh

Conventie

Binnen dit opleidingsonderdeel zullen `Linter` en `Prettier` steeds worden ingeschakeld bij het genereren van Vue-projecten.

Merk op dat er een extra commando getoond wordt in de CLI.

```
sh
Scaffolding project in /Users/barry/Projects/web-advanced/vue-extra...
|
└─ Done. Now run:

    cd vue-extra
    npm install
    npm run format
    npm run dev
```

Bij het bekijken van de `package.json` wordt opgemerkt dat zowel een `npm run format` als een `npm run lint`-script zijn toegevoegd.

Met `npm run format` gebruik je Prettier om de code te formatteren volgens de regels die in een configuratiebestand zijn gedefinieerd. Met `npm run lint` controleer je met Linter de code op mogelijke fouten en inconsistenties volgens de regels die in een configuratiebestand zijn gedefinieerd.

Bij het bekijken van de mapstructuur vallen enkele extra bestanden op.

```
plaintext
vue-extra/
- .vscode/
- node_modules/
- public/
- src/
. editorconfig
. gitattributes
. gitignore
oxlintrc.json
. prettierc.json
eslint.config.js
index.html
package.json
```

```
package-lock.json
README.md
vite.config.js
jsconfig.json
```

- **.gitattributes** : Bevat instellingen voor Git die bepalen hoe je bepaalde bestanden behandelt, zoals het instellen van tekstbestanden om altijd met LF-eindes te werken (Windows gebruikt CRLF, Mac en Linux gebruiken LF - dit kan problemen geven bij samenwerking wanneer dit niet consistent is).
- **.editorconfig** : Bevat instellingen voor code-editors om consistente code-stijlen te garanderen, zoals inspringing, einde van de regel (LF), ...
- **.prettierrc.json** : Configuratiebestand voor Prettier, waarin je de regels voor code-formattering definieert.
- **eslint.config.js** : Configuratiebestand voor de linter, waarin je de regels voor code-linting definieert.

Linter

Om te zien wat de Linter doet kan je een fout in de code introduceren.

Open **src/App.vue** en plaats onderstaande foutieve code in het **template** -gedeelte.

```
<h1></h2>
```

html

De editor toont een foutmelding op de **<h1>** -tag. Wanneer je over het rode lijntje met de muis beweegt, verschijnt een foutmelding waarin **eslint** wordt genoemd, dit is de linter die de code controleert op mogelijke fouten.

```
Parsing error: x-invalid-end-tag.eslintvue/no-parsing-  
error
```

plaintext

Verwijder de foutieve code zodat de code weer correct is.

Prettier

Om te zien wat Prettier doet kan je twee acties uitvoeren:

- Prettier uitvoeren via `npm run format`. Dit formatteert je code automatisch volgens de regels die in `.prettierrc.json` zijn gedefinieerd.
- De editor zo instellen dat Prettier automatisch je code formatteert bij het opslaan van een bestand.

Voordat je `npm run format` uitvoert, kan je een Git-repository in de map `vue-extra` initialiseren om te kunnen zien welke wijzigingen optreden na het uitvoeren van Prettier.

```
git init && git add -A && git commit -m "initial commit"
```

sh

Voer nu Prettier uit.

```
npm run format
```

sh

Indien geen wijzigingen worden gedetecteerd, kan het zijn dat de code al correct geformatteerd is. `.prettierrc.json` kan aangepast worden om zeker een wijziging te hebben.

Open `.prettierrc.json` en wijzig de `semi`-eigenschap van waarde `false` naar `true`. Dit zal ervoor zorgen dat Prettier automatisch een puntkomma toevoegt aan het einde van elke statement.

```
{  
  "$schema": "https://json.schemastore.org/prettierrc",  
  "semi": true,  
  "singleQuote": true,  
}
```

json

```
"printWidth": 100
}
```

Voer opnieuw Prettier uit.

```
npm run format
```

sh

```
npm run format
```

sh

```
> vue-extra@0.0.0 format
> prettier --write src/
```

```
src/App.vue 51ms
src/assets/base.css 5ms (unchanged)
src/assets/main.css 2ms (unchanged)
src/components/HelloWorld.vue 10ms
src/components/icons/IconCommunity.vue 2ms (unchanged)
src/components/icons/IconDocumentation.vue 1ms
(unchanged)
src/components/icons/IconEcosystem.vue 1ms (unchanged)
src/components/icons/IconSupport.vue 1ms (unchanged)
src/components/icons/IconTooling.vue 1ms (unchanged)
src/components/TheWelcome.vue 10ms
src/components/WelcomeItem.vue 4ms (unchanged)
src/main.js 1ms
```

In de console verschijnt welke bestanden je hebt gewijzigd en hoeveel tijd het kostte om elk bestand te formatteren. Ook in Git kan je de wijzigingen bekijken.

Om automatische formattering bij het opslaan van bestanden te voorzien, kan je de extensie **Prettier – Code formatter** in Visual Studio Code installeren.

Zodra je de extensie hebt geïnstalleerd, kan je de instellingen van Visual Studio Code aanpassen om Prettier automatisch uit te voeren bij het opslaan van een bestand.

Open de instellingen van Visual Studio Code (`Cmd + ,` op macOS of `Ctrl + ,` op Windows) en zoek naar `Format On Save`. Vink deze optie aan.

Nu zal Prettier automatisch je code formatteren telkens wanneer je een bestand opslaat.

De eerste keer dat je een bestand opslaat, kan er een melding verschijnen om de formatter te selecteren. Hierbij kies je voor `Prettier`.

`npm run format` is handig om alle bestanden in één keer te formatteren, bijvoorbeeld bij het wijzigen van een regel in `.prettierrc.json` of bij het automatisch formatteren van een heel project tijdens een `GitHub Action` (CI/CD).

Tijdens het werken in de editor is het handiger dat je de code automatisch formatteert bij het opslaan van een bestand. Op deze manier zorg je ervoor dat alle nieuwe wijzigingen steeds correct geformatteerd zijn.

Previous page
[Introductie](#)

Next page
[Oefeningen](#)